

Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Отчёт
По лабораторной работе N1
по дисциплине Информационная безопасность
Вариант: -

Работу выполнил:
Молчанов Федор Денисович Р3413

Работу приняла:
Маркина Татьяна Анатольевна

Ход работы

1. Разработка функционального API

Создан учебный REST API на Python без внешних runtime-зависимостей. Для HTTP слоя используется стандартный `http.server`, для хранения данных - SQLite.

Реализованы эндпоинты:

- `POST /auth/login` - проверяет логин и пароль, возвращает bearer-токен с ограниченным временем действия.
- `GET /api/data` - возвращает список публикаций, доступен только при наличии bearer-токена.
- `POST /api/posts` - создаёт новую публикацию от имени авторизованного пользователя.

Добавлены модульные тесты прикладной логики, работы с базой данных и механизма аутентификации. Поведение HTTP-эндпоинтов дополнительно проверено вручную с помощью `curl`.

2. Внедрение базовых мер защиты

Для защиты от SQL-инъекций все обращения к SQLite выполняются через параметризованные запросы с плейсхолдерами `?`. Пользовательский ввод не склеивается с SQL-строками.

Для снижения риска XSS все строковые значения в JSON-ответах проходят через `html.escape(..., quote=True)`. Это применяется централизованно перед отправкой ответа клиенту и дополнительно при возврате данных из методов приложения.

Механизм аутентификации переведён на JWT. После успешного `POST /auth/login` сервер выдаёт токен с `sub`, `username`, `iat` и `exp`, подписанный HMAC-SHA256. Защищённые методы проверяют заголовок `Authorization: Bearer <token>`.

Пароли не хранятся в открытом виде. Для тестового пользователя пароль хэшируется алгоритмом `scrypt` с солью; сравнение хэшей выполняется через `hmac.compare_digest`.

3. Настройка CI/CD pipeline с security-сканерами

В проект добавлен GitHub Actions workflow `.github/workflows/ci.yml`. Pipeline запускается при `push`, `pull_request` в ветку `main`, а также вручную через `workflow_dispatch`.

Workflow состоит из трёх независимых job:

- `Tests` - устанавливает dev-зависимости и запускает `pytest`.
- `SAST Bandit` - запускает Bandit по каталогу `infosec_rest`, формирует `bandit-report.json` и сохраняет его как artifact.
- `SCA OWASP Dependency-Check` - запускает OWASP Dependency-Check по корню проекта, формирует HTML/JSON/XML-отчёты и сохраняет их как artifact `dependency-check-report`.

Для SAST добавлен конфигурационный файл `bandit.yaml`, исключающий тесты из анализа. Для локального воспроизведения проверок добавлен `requirements-dev.txt`. На Arch Linux зависимости устанавливаются в локальное виртуальное окружение `.venv`, чтобы не изменять системный Python.

Публичный репозиторий проекта: github.com/ItmoSE/infosec-restapi. Последний успешный на момент подготовки отчёта запуск pipeline: GitHub Actions, запуск №34397112349.

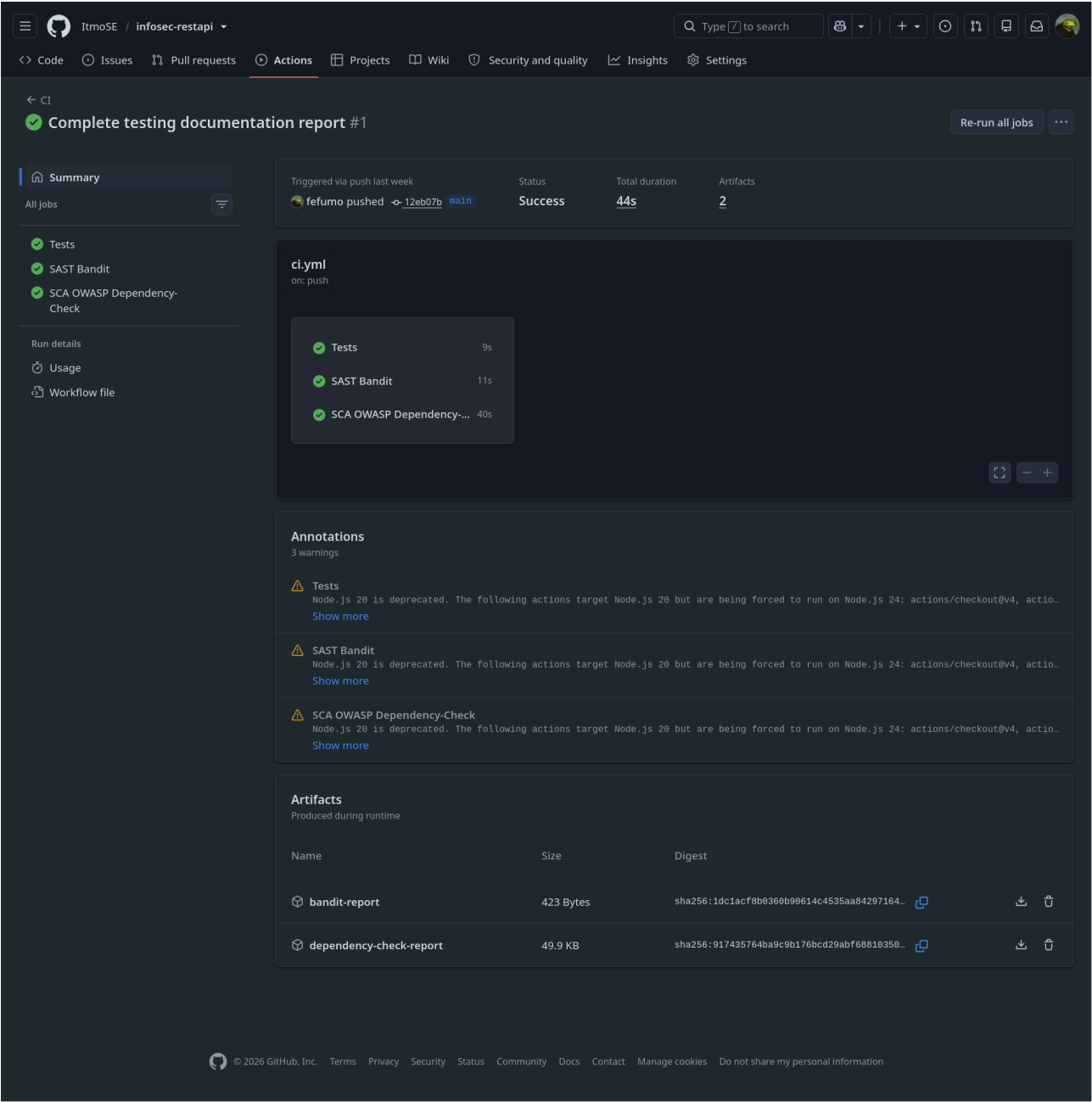


Figure 1: Успешный запуск CI pipeline и сформированные артефакты

ItmoSE / infosec-restapi

Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

CI

Complete testing documentation report #1

Re-run all jobs

Summary

All jobs

- Tests
- SAST Bandit**
- SCA OWASP Dependency-Check

Run details

- Usage
- Workflow file

Annotations

1 warning

SAST Bandit

succeeded last week in 11s

Search logs

- Set up job 1s
- Checkout repository 1s
- Set up Python 0s
- Install Bandit 4s
- Run Bandit 0s**
 - 1 ▶ Run bandit -r infosec_rest -c bandit.yaml -f json -o bandit-report.json
 - 11 [main] INFO profile include tests: None
 - 12 [main] INFO profile exclude tests: Bi01
 - 13 [main] INFO cli include tests: None
 - 14 [main] INFO cli exclude tests: None
 - 15 [json] INFO JSON output written to file: bandit-report.json
- Upload Bandit report 1s
- Post Set up Python 0s
- Post Checkout repository 1s
- Complete job 0s

Figure 2: Успешное выполнение SAST-проверки Bandit

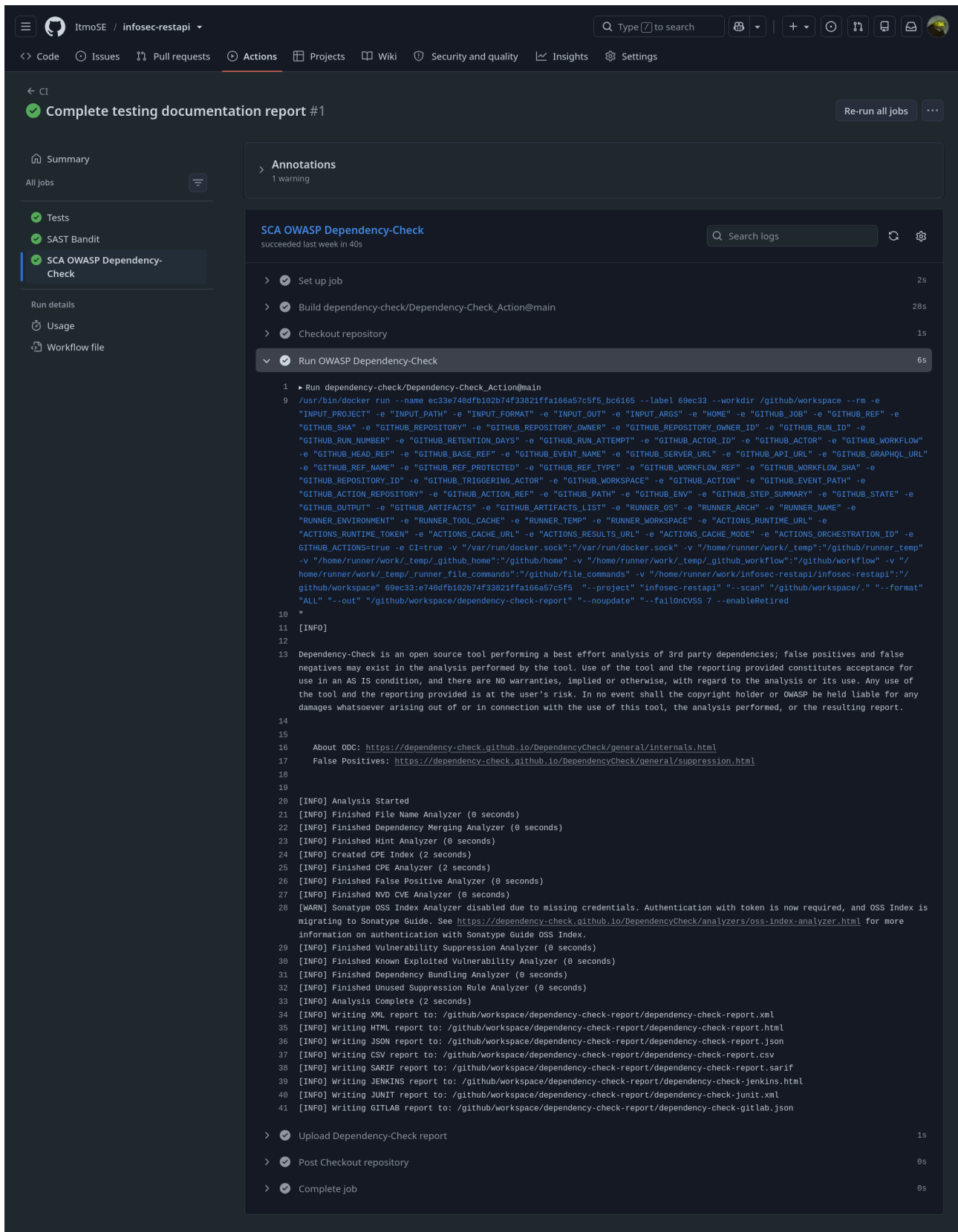


Figure 3: Успешное выполнение SCA-проверки OWASP Dependency-Check

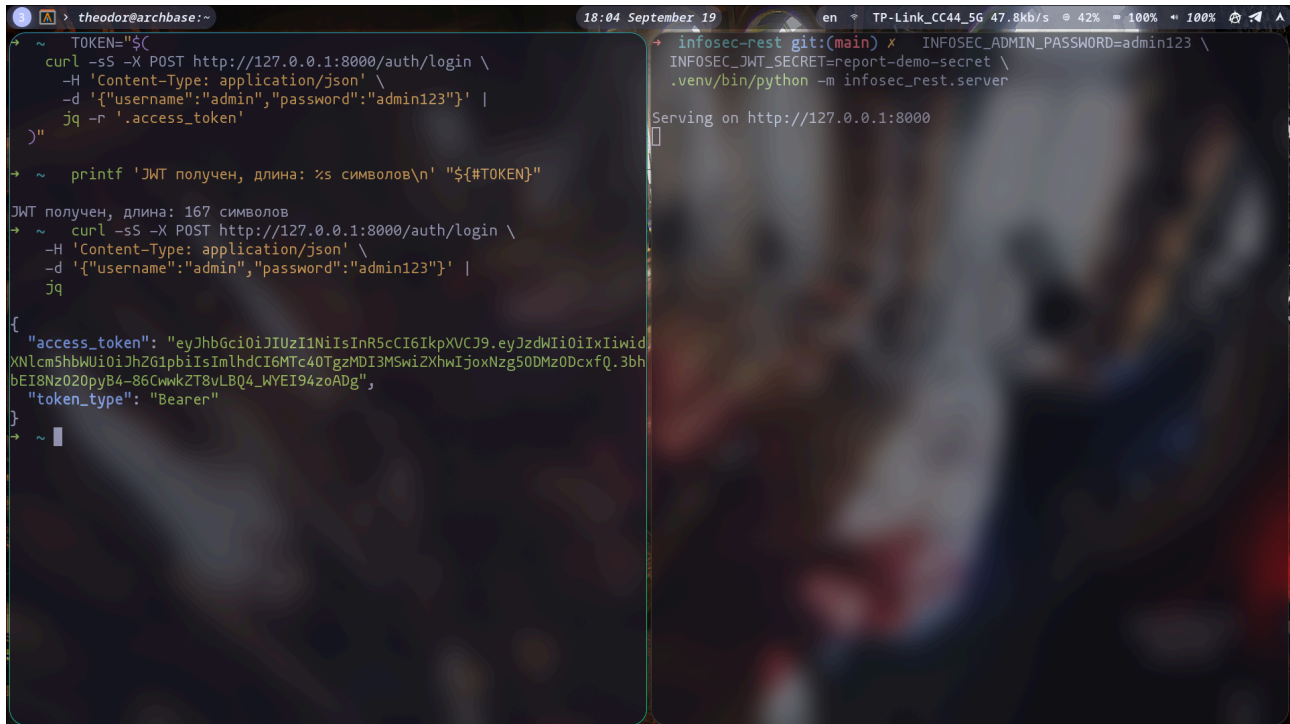
4. Тестирование и документирование

Работа API проверяется автоматическими тестами `pytest`. Тесты покрывают: успешную аутентификацию, отказ при неверном пароле, защиту от SQL-инъекции при логине, хранение пароля в виде `bcrypt`-хэша, миграцию старой схемы, создание публикации, экранирование пользовательского HTML и отклонение подделанного JWT.

Для ручной проверки используются команды curl, приведённые в README: получение токена через POST /auth/login, обращение к GET /api/data с токеном и проверка отказа без заголовка Authorization.

Локально выполнены проверки:

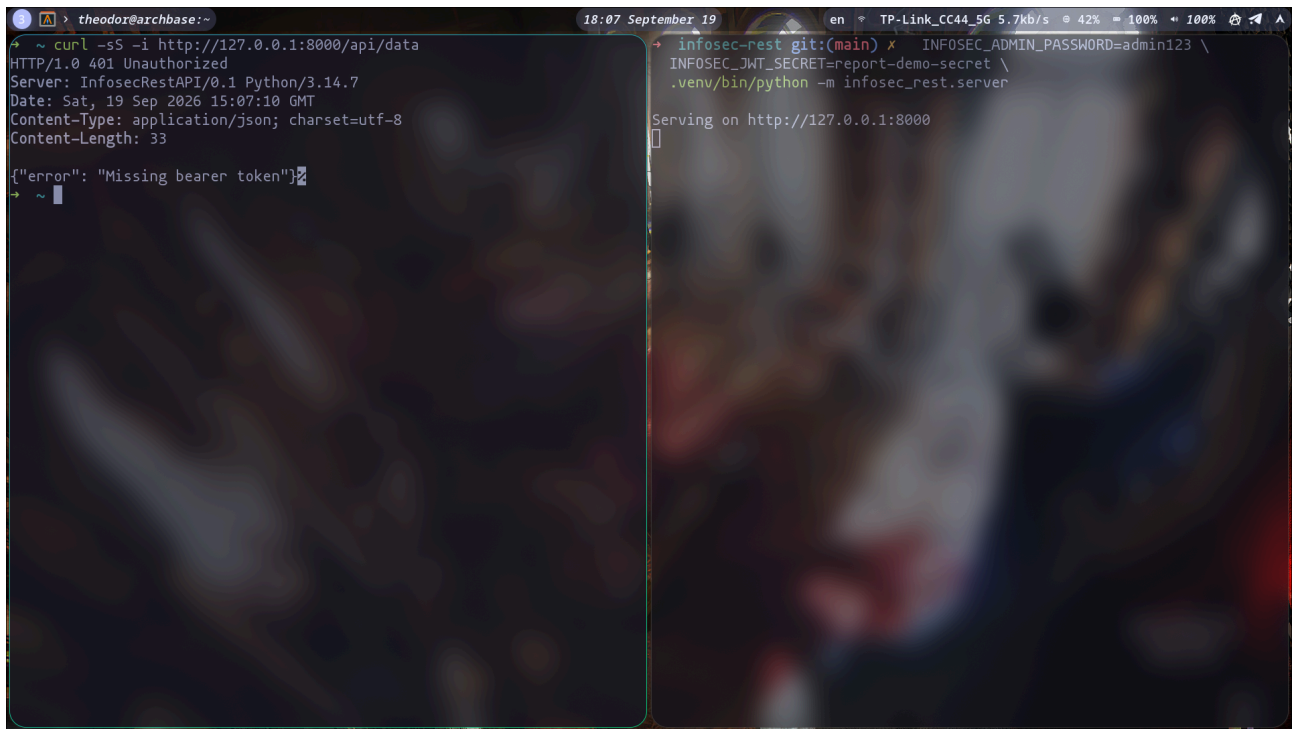
- .venv/bin/python -m pytest - 9 тестов пройдены успешно.
- .venv/bin/bandit -r infosec_rest -c bandit.yaml - проблем не найдено.
- GET /health - получен 200 OK.
- GET /api/data без токена - получен 401 Unauthorized.
- POST /auth/login с тестовой учётной записью - получен JWT.
- GET /api/data с Authorization: Bearer <token> - получен 200 OK и список публикаций.
- POST /api/posts с JWT - публикация создана, получен 201 Created.
- Попытка SQL-инъекции в поле username отклонена с 401 Unauthorized.
- Пользовательские HTML-теги возвращены в экранированном виде.



```
theodor@archbase:~$ TOKEN=$(curl -sS -X POST http://127.0.0.1:8000/auth/login \
-H 'Content-Type: application/json' \
-d '{"username":"admin","password":"admin123"}' |
jq -r '.access_token')

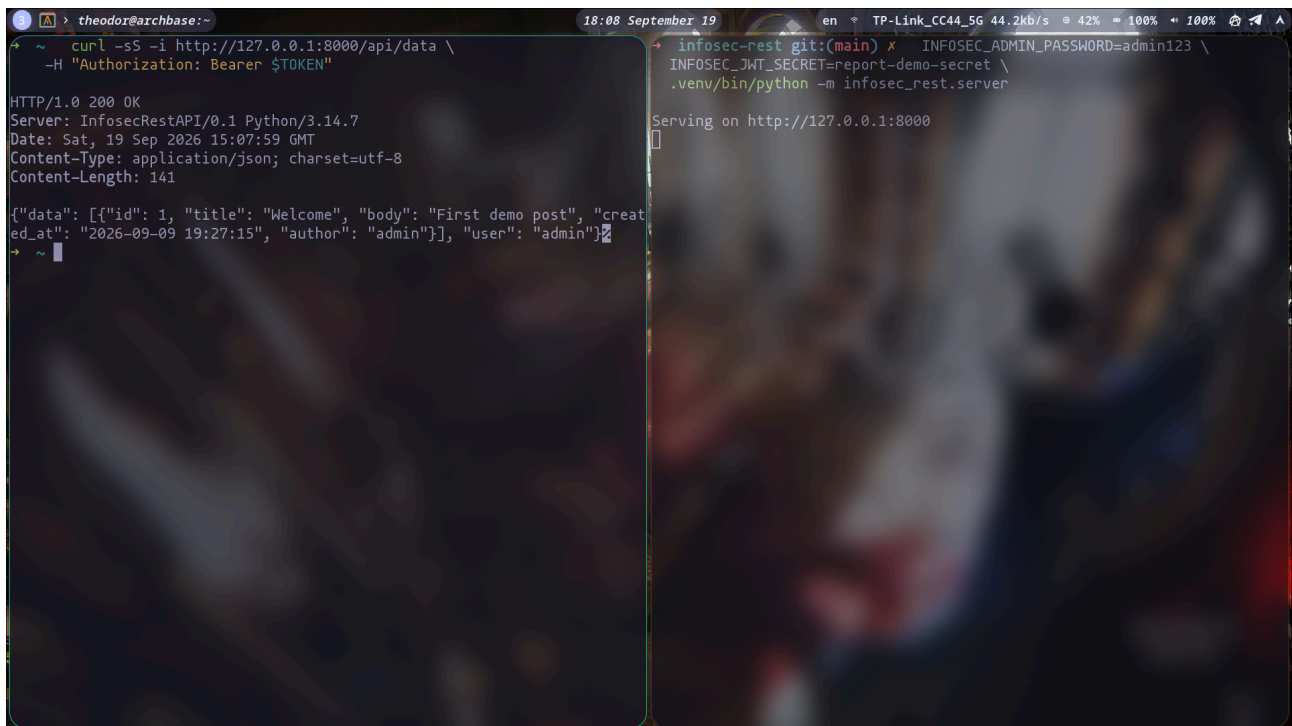
~$ printf 'JWT получен, длина: %s символов\n' "${#TOKEN}"
JWT получен, длина: 167 символов
~$ curl -sS -X POST http://127.0.0.1:8000/auth/login \
-H 'Content-Type: application/json' \
-d '{"username":"admin","password":"admin123"}' |
jq
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxIiwiaWF0IjoiMTY5ODQ0MzU0LmZ0dCxfQ.3bh",
  "token_type": "Bearer"
}
```

Figure 4: Проверка получения JWT через curl



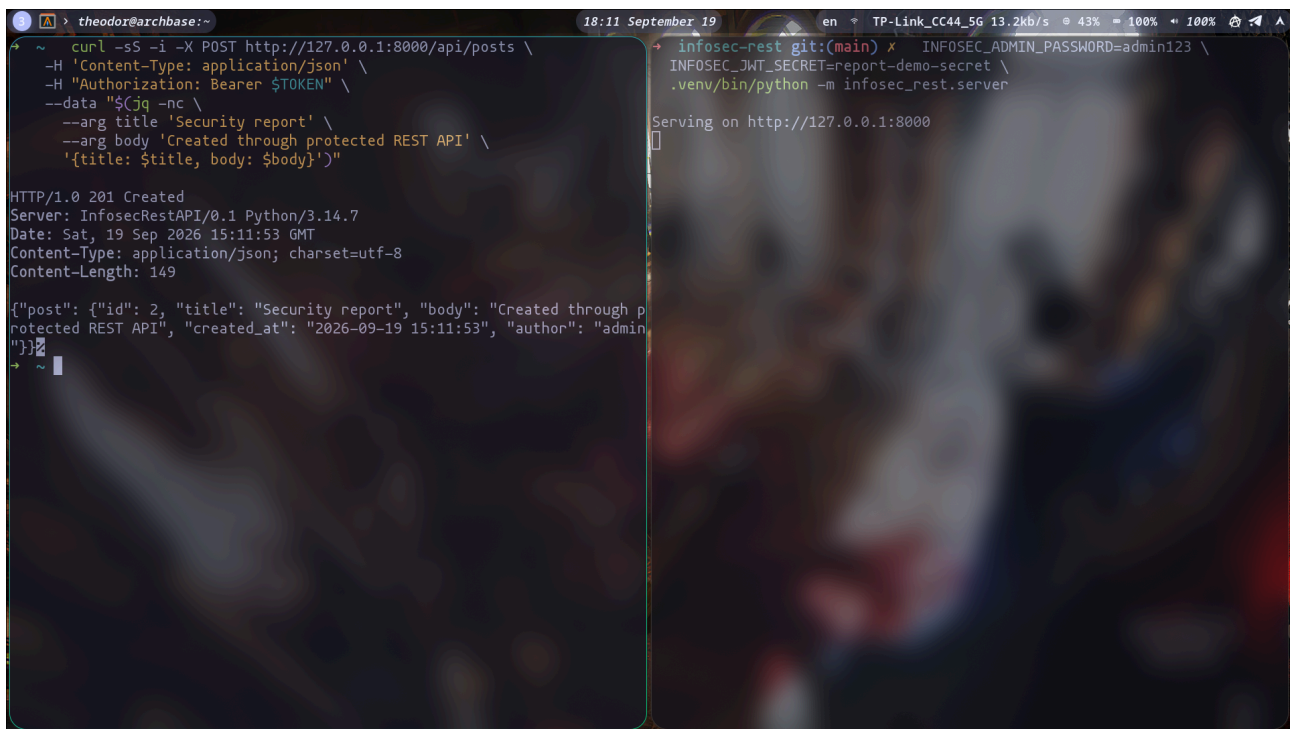
```
theodor@archbase:~  
~ curl -sS -i http://127.0.0.1:8000/api/data  
HTTP/1.0 401 Unauthorized  
Server: InfosecRestAPI/0.1 Python/3.14.7  
Date: Sat, 19 Sep 2026 15:07:10 GMT  
Content-Type: application/json; charset=utf-8  
Content-Length: 33  
  
{"error": "Missing bearer token"}  
~
```

Figure 5: Проверка запрета доступа без JWT



```
theodor@archbase:~  
~ curl -sS -i http://127.0.0.1:8000/api/data \\  
-H "Authorization: Bearer $TOKEN"  
HTTP/1.0 200 OK  
Server: InfosecRestAPI/0.1 Python/3.14.7  
Date: Sat, 19 Sep 2026 15:07:59 GMT  
Content-Type: application/json; charset=utf-8  
Content-Length: 141  
  
{"data": [{"id": 1, "title": "Welcome", "body": "First demo post", "creat  
ed_at": "2026-09-09 19:27:15", "author": "admin"}], "user": "admin"}  
~
```

Figure 6: Проверка доступа к защищённому endpoint с JWT



```
theodor@archbase:~  
18:11 September 19  
en TP-Link_CC44_5G 13.2kb/s 43% 100% 100%  
~ curl -sS -i -X POST http://127.0.0.1:8000/api/posts \  
-H 'Content-Type: application/json' \  
-H 'Authorization: Bearer $TOKEN' \  
--data "$(jq -nc \  
--arg title 'Security report' \  
--arg body 'Created through protected REST API' \  
'{title: $title, body: $body}')"  
  
HTTP/1.0 201 Created  
Server: InfosecRestAPI/0.1 Python/3.14.7  
Date: Sat, 19 Sep 2026 15:11:53 GMT  
Content-Type: application/json; charset=utf-8  
Content-Length: 149  
  
{ "post": { "id": 2, "title": "Security report", "body": "Created through p  
rotected REST API", "created_at": "2026-09-19 15:11:53", "author": "admin  
"} }  
~
```

Figure 7: Создание публикации через защищённый endpoint



```
theodor@archbase:~  
18:17 September 19  
en TP-Link_CC44_5G 43.4kb/s 43% 100% 100%  
~ curl -sS -i -X POST http://127.0.0.1:8000/auth/login \  
-H 'Content-Type: application/json' \  
-d '{"username": "admin' OR 1=1 --", "password": "anything"}'  
  
HTTP/1.0 401 Unauthorized  
Server: InfosecRestAPI/0.1 Python/3.14.7  
Date: Sat, 19 Sep 2026 15:16:52 GMT  
Content-Type: application/json; charset=utf-8  
Content-Length: 32  
  
{ "error": "Invalid credentials" }  
~
```

Figure 8: Отклонение попытки SQL-инъекции при аутентификации



```
theodor@archbase:~$ curl -s -X POST http://127.0.0.1:8000/api/posts \
-H 'Content-Type: application/json' \
-H "Authorization: Bearer $TOKEN" \
-d '{"title": "<script>alert(1)</script>", "body": "<b>XSS test</b>"}' |
jq
{
  "post": {
    "id": 3,
    "title": "&lt;script&gt;alert(1)&lt;/script&gt;",
    "body": "&lt;b&gt;XSS test&lt;/b&gt;",
    "created_at": "2026-09-19 15:15:34",
    "author": "admin"
  }
}
```

```
infosec-rest git:(main) x INFOSEC_ADMIN_PASSWORD=admin123 \
INFOSEC_JWT_SECRET=report-demo-secret \
.venv/bin/python -m infosec_rest.server
Serving on http://127.0.0.1:8000
```

Figure 9: Экранирование пользовательских HTML-данных в JSON-ответе

Ответы на контрольные вопросы

1. Почему bcrypt или scrypt предпочтительнее SHA-256 для паролей?

SHA-256 спроектирован как быстрый алгоритм, поэтому злоумышленник может проверять большое количество вариантов пароля в секунду. Алгоритмы bcrypt и scrypt намеренно требуют больше ресурсов и поддерживают настраиваемую вычислительную стоимость. Кроме того, при хэшировании применяется уникальная соль, поэтому одинаковые пароли не дают одинаковых сохранённых значений. scrypt также требует значительного объёма памяти, что усложняет массовый перебор на GPU и специализированном оборудовании.

2. В чём разница между SAST и DAST?

SAST анализирует исходный код или байткод приложения без необходимости запускать его и позволяет связать найденную проблему с конкретным участком кода. DAST исследует уже запущенное приложение снаружи, отправляя запросы и анализируя ответы. Поэтому DAST учитывает фактическую конфигурацию приложения, но обычно не указывает точное место ошибки в исходном коде. Подходы дополняют друг друга.

3. Как работает JWT?

JWT состоит из трёх частей: заголовка, полезной нагрузки и подписи. В данном проекте заголовок указывает алгоритм HS256, а полезная нагрузка содержит идентификатор пользователя sub, имя пользователя, время выдачи iat и срок действия exp. Сервер вычисляет HMAC-SHA256 от заголовка и нагрузки с помощью секретного ключа и сравнивает полученную подпись с подписью токена. Также проверяется срок действия. Полезная нагрузка JWT кодируется, но не шифруется, поэтому в неё нельзя помещать конфиденциальные данные.

4. Чем опасно отсутствие аудита сторонних библиотек?

Зависимости могут содержать известные уязвимости, включая уязвимости во вложенных транзитивных зависимостях. Использование устаревших или скомпрометированных пакетов создаёт риск выполнения произвольного кода, утечки данных и нарушения доступности сервиса, даже если собственный код приложения написан безопасно. SCA-инструменты сопоставляют используемые компоненты с базами известных уязвимостей и позволяют своевременно обновлять их.

Вывод

В ходе работы разработан REST API с JWT-аутентификацией, параметризованными SQL-запросами, экранированием пользовательских данных и хранением паролей в виде `bcrypt`-хэшей. В GitHub Actions интегрированы автоматические тесты, SAST Bandit и SCA OWASP Dependency-Check. Ручные проверки подтвердили выдачу JWT, ограничение доступа к защищённым endpoint, создание публикаций и работу мер защиты от SQL-инъекций и XSS. Настроенный pipeline завершился успешно и сформировал отчёты обоих security-сканеров.